

附件 4

2026 年安徽省 AI 大模型创新应用竞赛

作品赛赛道

作品名称：知行（ZHIXING AGENT）

2026 年 9 月

填写说明

1. 所有参赛作品必须为一个基本完整的设计。作品报告书旨在能够清晰准确地阐述（或图示）该参赛队的参赛作品（或方案）。
2. 作品报告采用 A4 纸撰写。除标题外，所有内容必需为宋体、五号字、1.5 倍行距。
3. 作品报告中各作品模板文字部分仅供参考，作品报告书撰写完毕后，请删除所有模板文字。（本页不删除）
4. 作品报告模板里已经列的内容仅供参考，作者可以在此基础上增加内容或对文档结构进行微调。
5. 网评采用双盲评审，因此请勿在该文档出现任何有关学校、队伍、个人等信息，否则按作弊论处。
6. 报告正文不超过 10 页。
7. 作品最终以 PDF 格式提交。

作品简介

作品名称	知行（ZHIXING AGENT）
主题	主题 1：大模型驱动的智能交互与对话系统
作品简介	知行是一套面向校园与日常生活场景的多用户 Web 智能体交互系统。系统以大模型对话为统一入口，结合 ReAct 决策与工具调用，根据用户意图选择天气、课表、考试、旅行、地图、文件和目标跟进等技能，通过 SSE 返回流式回答与任务状态。多轮上下文、长期记忆和任务续接支撑连续交互；图片、音频和文件处理扩展任务输入与输出。项目以智能交互与对话系统为主体，以知识记忆和多模态能力为支撑，将需求理解、工具执行、结果反馈和后续提醒接入同一工作台。
作品来源	面向个人事务管理与多模态交互需求设计开发的 Web 智能助理应用。
应用行业	个人效率工具、校园生活服务、日常出行与信息管理等。
数据来源	用户主动提交的对话与附件；公开信息接口；Skill 配置与领域知识；模型生成及向量化结果。
作品展示	浏览器端 Web 程序；支持文本、图片、音频与文件交互。

一 选题定位与社会价值

知行选择“主题 1：大模型驱动的智能交互与对话系统”。作品面向校园与日常生活，以浏览器中的多用户对话工作台为入口：用户描述目标，系统结合上下文识别意图、补齐条件并选择技能与工具，再反馈执行结果。设计重点是让对话能够推进实际任务，而不只是生成建议。知识记忆用于保留背景与偏好，多模态能力用于接收资料和交付产物，两者共同支撑连续交互。

1 需求与问题

很多日常任务不是问一句就结束。安排出行要查天气、地点和路线；导入课表要先读文件，再确认课程信息和提醒时间；长期目标还要在之后的对话中继续记录。普通聊天模型给出建议后，剩下的操作仍由用户完成。知行把这些步骤放进同一个对话工作台，让模型的判断能接到实际工具和后续任务上。

2 目标用户与场景

学生可以上传课表或考试资料，让知行解析、查询并设置提醒。准备出行时，用户可以在一次对话里查询天气、地点、路线和交通方案。遇到文档、图片或音频资料，也可以直接上传处理。目标、定时任务和信息订阅则用来承接不能在当轮结束的事情。这些功能共用账户、历史对话和工具轨迹，用户不必为每个场景重新学习一套操作方式。

3 预期价值与推广条件

知行把模型生成的内容和真实操作放在一起。查询结果、生成文件和后续通知都会回到 Web 工作台；记忆页面允许用户查看已经保存的内容；工具轨迹会显示系统调用了什么能力。删除或修改文件等高风险操作会先出现确认卡片。新增领域能力时，可以增加 Skill 配置和工具适配，不需要重写整条对话链路。

目前的版本适合个人使用和小规模试用。要扩大使用范围，还需要解决模型和第三方接口的稳定性、通知投递以及日常运维问题。项目尚未积累足够的真实用户数据，因此报告不预设效率或成本收益，后续将通过任务完成率、响应时间和用户反馈进行测量。

二 系统架构与模型分工

前端使用 React 18、TypeScript 和 Vite，后端使用 Java 21 与 Spring Boot 3.3。构建后的前端文件由 Spring Boot 直接托管，因此部署时仍是一个 Web 应用。图中按代码职责划分模块，用来说明调用关系；这些模块并不是独立部署的微服务。

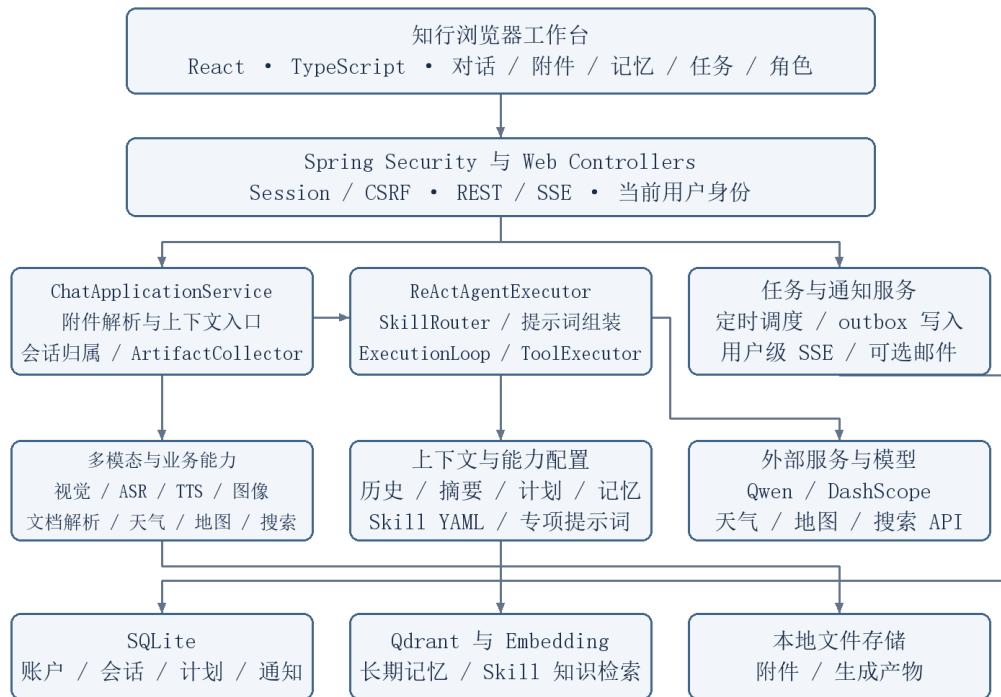


图 1 知行技术架构。聊天与定时任务共享业务能力；通知服务写入 outbox 后调用用户级 SSE 推送。模型、向量检索及第三方信息接口属于外部依赖。

1 模型负责理解与生成

配置模板将文本模型设为 qwen3.6-plus，视觉模型为 qwen-vl-max，图像模型为 qwen-image-2.0，ASR 为 paraformer-v2，TTS 为 cosyvoice-v3-flash。文本模型通过兼容接口返回正文或结构化工具调用，业务代码负责实际执行。各模型通过配置接入，部署时可根据服务可用性与成本调整；运行前需分别检查对应接口。

2 存储负责状态与结果

SQLite 保存账户、可见消息、模型上下文、运行状态和业务数据；Qdrant 支撑长期记忆与 Skill 知识检索；ArtifactService 管理上传件与生成产物。长期记忆的 Embedding 示例配置使用 BGE-M3、1024 维；向量模型与知识集合维度需要保持一致。

三 从输入到执行的核心流程

对话主链路为 WebChatController → ChatApplicationService → ReActAgentExecutor → SkillRouter / ExecutionLoop / Tools。入口校验用户与会话归属、建立 runId 并解析附件；执行器组织技能路由、上下文和动态工具集，再由 ReAct 循环依据工具结果继续决策或结束回复。

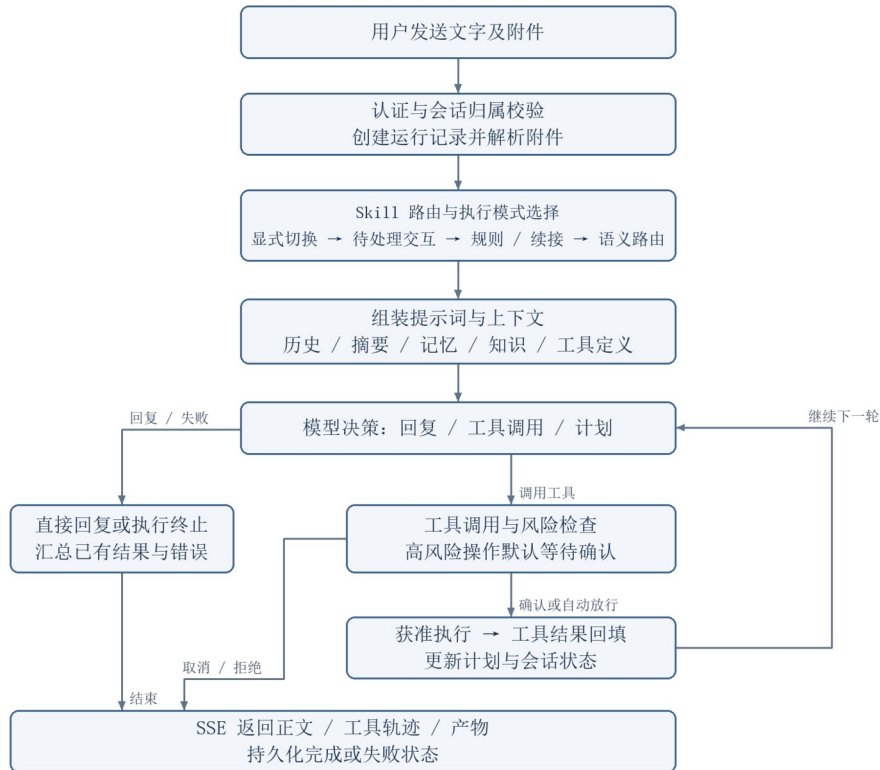


图2 内联 ReAct 主执行流程。专用 Skill 执行器可承接独立工作流；已有待确认交互也可在进入普通循环前处理。工具结果回到模型，构成后续判断依据。

这条链路对应“理解需求—规划执行—工具协同—结果反馈”。用户补充条件或调整要求时，系统结合会话状态继续处理；信息不足时追问，高风险操作等待确认。SSE 将回答和工具状态推送到前端，用户可以核对调用过程、失败位置和生成产物。交互因此既包含自然语言交流，也包含对执行过程的确认与修正。

SkillRouter 按显式切换、待处理交互、强规则触发、续接判断及语义路由选择能力。ExecutionLoop 默认限制最多 15 轮、累计 32 次工具调用，并处理空回复、结构化计划校验和终止汇总。SafetyPolicy 默认对 file_delete 与 file_update 要求确认；用户可确认或取消；其他工具依据风险规则直接执行或记录日志。

四 提示词与多模态上下文设计

1 分层提示词与能力约束

发送给文本模型的内容不是一段固定提示词。系统规则先限定角色和执行要求，当前 Skill 再补充旅行、校园或天气等场景的处理方式，工具定义告诉模型可以调用什么、参数怎样填写。需要领域知识时，SystemPromptBuilder 将检索结果加入系统提示；DefaultContextBuilder 不再重复检索同一批知识。

以“根据这份课表安排提醒”为例，系统先读取附件，再识别课程和提醒条件。如果时间、学期等信息不完整，就继续向用户询问；信息齐全后才调用课表或提醒工具。系统判断任务是否完成时会检查工具结果和业务状态，不能只接受模型生成的一句“已经设置”。SkillReplyGuard 用于约束提醒、目标等场景的结束回复。

2 历史摘要与长期记忆

DefaultContextBuilder 组合最近对话轮次、当前消息与长期记忆，并支持摘要和按轮次边界的预算裁剪。当前 ReActAgentExecutor 构造时将预算参数传为 0，即该入口尚未启用固定 token 预算裁剪。后续可依据实际上下文用量评估并启用合适的预算。

对于“记住……”类明确请求，ChatApplicationService 还提供异步保存路径：匹配短文本后调用 saveManual。该路径降低了对模型主动调用记忆工具的依赖，但仍可能因存储或外部依赖异常失败，因此不承诺记忆绝对不会丢失。长期记忆与会话摘要分别服务于跨轮偏好和历史压缩，职责不同。

3 附件输入与产物输出

附件先校验用户归属，再按类型处理：图片转为视觉请求，音频转写为文本，文件交由解析服务提取内容。提取结果以附件标识拼接到输入消息；上传处理去重并最多取 8 件附件。文档解析依赖 Tika、PDFBox 和 POI 等组件，与调用视觉模型属于不同处理路径。

生成图片、语音或文件通过请求级 ArtifactCollector 汇入当轮响应，由前端展示或下载。知行同时提供角色管理界面及音色相关配置；预设音色、样本存档和自定义声音合成应按各自接口条件分别配置与验证。

五 工程特点与技术问题

1 规则与模型联合路由

SkillRouter 没有把所有意图判断都交给模型。用户明确切换能力、回复待确认问题或命中高置信度规则时，系统先按确定路径处理；只有这些条件不足时，才使用语义路由。这样做是为了避免多轮对话继续沿用已经失效的场景。选中的 Skill 会缩小本轮可用能力范围，使基础模型的判断落到具体任务步骤上。

交互层将工具调用状态、待确认操作和业务结果呈现给用户，使用户能够参与执行过程。可核对的工具记录是判断任务完成的依据，不把模型生成的解释本身当作执行成功的证明。

2 可见记录与模型上下文解耦

chat_messages 和 chat_runs 保存用户可见正文、附件、工具轨迹、产物及状态；context_messages 服务于模型上下文。两者分离，使界面记录不必完全依赖于推理的历史格式。启动迁移会将未完成的 STREAMING 运行标记为 FAILED，并保留中断前内容；这属于结果保留和失败恢复，不等同于自动续跑所有任务。

3 身份传递与操作确认

Spring Security 使用会话认证、CSRF 和 BCrypt。用户身份传递到会话、记忆、附件和通知服务，关键读写使用用户归属条件；ArtifactService 加入文件访问边界。租户隔离是已实现的设计机制，仍应结合完整接口测试检查覆盖情况。默认高风险工具确认、取消与失败状态在前端以轨迹和卡片反馈。

4 主动服务闭环

定时任务、课程提醒及订阅服务将通知交给 DatabaseNotificationSink。服务保存 notification_outbox 后直接调用 NotificationStreamService.notify，向该用户在线连接推送；历史和未读记录通过查询接口读取。邮件是按用户设置启用的旁路渠道，并非仅在检测到离线时才发送。邮件异常被隔离处理，避免其失败直接中断站内通知逻辑。

5 当前边界与改进方向

系统仍受外部服务影响：文本模型或地图、天气接口不可用时，相应任务无法完整执行；SSE 断开后，用户可能收不到当时的实时事件；SQLite 也不适合在未评估的情况下直接承担更高并发。通知目前在数据库事务中写入记录后立即推送，还没有独立派发器和可靠重试。下一步应先补外部依赖监测、通知提交后投递和端到端场景测试。

六 验证设计与演示安排

代码库已经为 Skill 路由、工具确认、上下文预算、附件归属、会话记录、通知、课程解析和执行循环编写测试。这些测试主要检查单个模块和关键边界。竞赛演示还需要按完整用户任务记录成功或失败、响应时间以及失败发生在哪一步，才能评价系统在真实流程中的表现。

1 场景演示

演示一：上传课表，补充学期与提醒条件，检查追问、解析和查询是否衔接。演示二：提出天气与路线需求，再修改目的地，检查技能切换、上下文续接及工具结果。演示三：保存个人偏好，在记忆页核对后开启新一轮对话，检查召回是否相关。

演示四：请求修改或删除一个测试文件，确认界面先出现待确认状态，分别演示取消与允许。演示五：创建短周期测试提醒，在通知中心检查消息；启用邮件时检查旁路投递。演示六：上传图片或音频，核对视觉描述、转写与后续处理是否保持语义一致。上述场景组成后续演示与验收计划，各项结果应留存运行记录和界面证据。

2 建议评测指标

任务成功率应以业务状态变化和有效产物为标准；延迟记录从请求到首段可见正文及最终完成的时间；记忆评测分别检查保存、召回与无关内容误注入；确认机制检查未授权执行比例；租户隔离以两个测试账户交叉访问验证；通知区分数据库持久化、在线推送和邮件投递结果。

3 运行与交付

本地运行需要 Java 21、前端构建环境、SQLite 文件目录，以及所启用的模型、

Embedding、Qdrant 和第三方接口配置。前端构建后由 Spring Boot 托管。演示数据宜使用不含真实身份信息的样例，模型密钥通过环境变量或本地配置管理，提交材料不附带真实密钥和个人数据。

知行以大模型驱动的智能交互系统为主体，以知识记忆和多模态为支撑。后续优先验证多轮续接、工具执行、确认交互和主动提醒的完成质量，再扩展场景。